

COMP 576 Assignment 2

Lingyi Xu (lx28@rice.edu)

Nov 5th, 2024

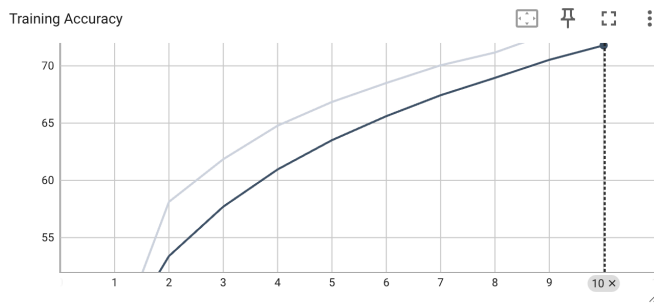
Table of Contents

- COMP 576 Assignment 2
 - Table of Contents
 - 1. Visualizing a CNN with CIFAR10
 - 1. b) Train LeNet5 on CIFAR10
 - 1. c) Visualize the Trained Network
 - 2. Visualizing and Understanding Convolutional Networks
 - Summarize the key ideas of the paper
 - (Optional) Visualization of features in a fully trained model
 - 3. Build and Train an RNN on MNIST
 - 3. a) Setup an RNN
 - 3. b) How about using an LSTM or GRU
 - LSTM Results:
 - GRU Results:
 - Accuracy Comparison:
 - Loss Comparison:
 - Change the number of hidden units
 - 3. c) Compare against the CNN

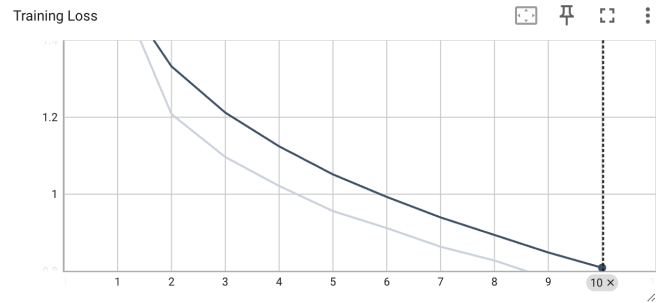
1. Visualizing a CNN with CIFAR10

1. b) Train LeNet5 on CIFAR10

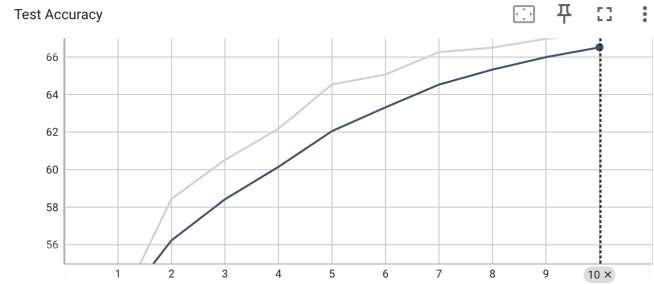
Initially, I used a learning rate of 0.001 and the Adam optimizer as default settings. Here are the train/test accuracy and train loss plots generated:



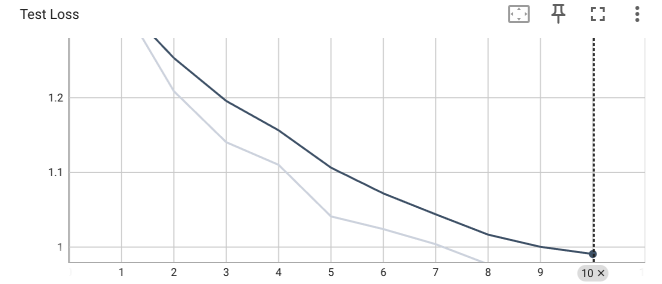
Run	Smoothed Value	Step	Time	Value	Relative	Step	Relative
Nov03_19-17-45	71.79	73.66	10	11/3/24, 7:34 PM	14.69 min	10	14.69 min



Run	Smoothed Value	Step	Time	Value	Relative	Step	Relative
Nov03_19-17-45	0.8089	0.75	10	11/3/24, 7:34 PM	14.69 min	10	14.69 min



Run	Smoothed Value	Step	Time	Value	Relative	Step	Relative
Nov03_19-17-45	66.52	67.29	10	11/3/24, 7:34 PM	14.7 min	10	14.7 min



Run	Smoothed Value	Step	Time	Value	Relative	Step	Relative
Nov03_19-17-45	0.9907	0.9764	10	11/3/24, 7:34 PM	14.7 min	10	14.7 min

The results obtained are as follows:

- Test Accuracy = 67.29
- Test Loss = 0.9764
- Training Accuracy = 73.662
- Training Loss = 0.75

Next, with epochs = 5, different optimizers, learning rates, and momentum values are experimented. The results for train/test accuracy and train loss across various configurations are summarized in the table below:

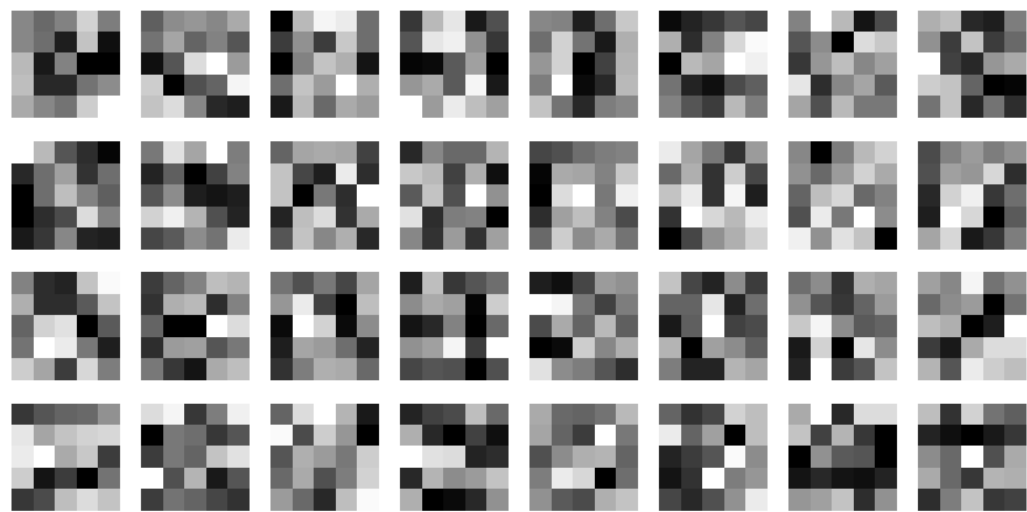
No.	Optimizer	Learning Rate	Momentum	Test Accuracy	Test Loss	Training Accuracy	Training Loss
1	Adam	0.001	-	64.54	1.0411	66.85	0.9561
2	SGD	0.001	-	23.93	2.1566	22.886	2.1791
3	Adam	0.01	-	16.15	2.4999	16.15	2.6682
4	Adam	0.005	-	10.00	2.3034	10.08	2.3037
5	SGD	0.01	-	44.98	1.5635	43.63	1.5963
6	SGD	0.005	-	37.04	1.8221	35.19	1.8672
7	SGD	0.001	0.8	36.98	1.8251	35.11	1.8694

No.	Optimizer	Learning Rate	Momentum	Test Accuracy	Test Loss	Training Accuracy	Training Loss
8	SGD	0.001	0.5	28.09	2.0514	27.22	2.0659

Summary: Using the **Adam** optimizer with a **learning rate of 0.001** provided the best overall model performance. This configuration achieved a balanced performance in terms of both training and test accuracy as well as loss, making it an ideal choice for the model.

1. c) Visualize the Trained Network

Using a learning rate of 0.001, 10 epochs, and the Adam optimizer, the visualization of the first convolutional layer’s weights is shown below:



The statistics of the activations in the convolutional layers on test images are as follows:

Activation mean: `-0.07476064562797546`, variance: `0.03350016847252846`

The mean value is close to 0, which indicates that the activations in the convolutional layer are relatively balanced. This distribution is typical and beneficial as it helps prevent gradient vanishing or exploding during training. The variance of 0.0335 indicates that the activations are spread within a narrow range, suggesting that the layer's responses to inputs have stabilized after training.

2. Visualizing and Understanding Convolutional

Networks

Summarize the key ideas of the paper

The paper *Visualizing and Understanding Convolutional Networks* by Matthew D. Zeiler and Rob Fergus's core goal is to help us understand how Convolutional Neural Networks (CNNs) function internally and to improve these models through this understanding. CNNs have achieved impressive performance on image classification tasks, especially on large datasets like ImageNet, but their inner workings are complex, and it's hard to understand how exactly they operate. This paper introduces an innovative visualization technique that reveals how features are activated in different layers of CNNs, helping us design and optimize models more effectively.

Visualization Technique

The paper introduces a technique called Deconvolutional Network (DeconvNet), which maps activations within a CNN back to the input image. In other words, when a specific feature in a certain layer is activated, this technique “traces back” to the input pattern that caused this activation. This method allows us to visualize the features learned in each layer and observe how these features evolve during training, revealing the hierarchical structure of features inside CNNs.

Hierarchical Feature Learning

Each layer in a CNN learns features of increasing complexity, forming a hierarchical structure. The lower layers detect simple edges and color patterns, the middle layers capture textures and shapes, and the higher layers represent high-level, class-specific features like parts of an object (such as a face or an eye of a dog). This hierarchy explains why CNNs are effective: they can progressively capture more meaningful and complex features.

Improving Architecture

By examining the visualizations of CNNs, the researchers identified some issues, such as missing low-frequency features and “aliasing effects” in certain layers. By reducing the filter size in the first layer (from 11x11 to 7x7) and lowering the stride (from 4 to 2), they significantly improved the model's feature representation and classification performance. This demonstrates that visualization is not only a tool for understanding models but also for optimizing architectures to enhance performance.

Analyzing with Occlusion Experiments

The researchers conducted occlusion experiments, systematically covering parts of input images to see if the model was truly focusing on the object itself rather than relying on background or other context. The results showed that the model's accuracy dropped significantly when key parts of the object were obscured, indicating that it genuinely focuses on the local structure of the object and not just on scene context.

Model Depth and Transfer Learning

In studying the contribution of each layer, the authors also conducted an Ablation Study by removing or adjusting specific layers to analyze their impact on performance. Results showed that depth is crucial to CNNs—deeper networks are better at capturing complex features, ultimately leading to better performance. Additionally, they tested this ImageNet-trained model on other datasets (such as Caltech-101 and Caltech-256) and found that its features generalized well, demonstrating that CNNs can reuse features across tasks, making them well-suited for transfer learning.

In conclusion, the paper shows that visualization techniques help us understand and debug models, pointing to potential areas of improvement. The progressive abstraction of features in different layers allows CNNs to recognize complex objects by starting from simple edges, which can contribute to the development of CNN architectures and help future models to learn and transfer useful features more effectively.

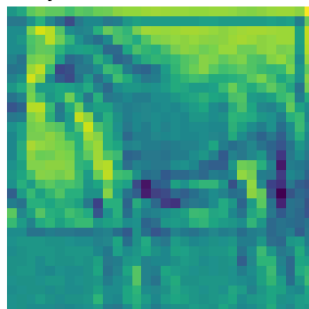
(Optional) Visualization of features in a fully trained model

Apply the DeconvNet technique to visualize features learned in the first convolutional layer of our CNN model trained on the CIFAR-10 dataset. For each feature, we examined:

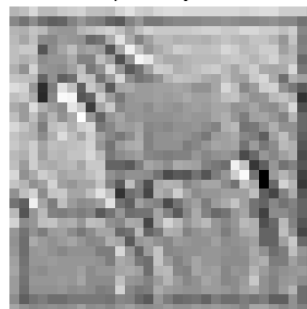
1. Activation Map: The response in a specific feature map when a test image is passed through the model.
2. Deconvolution Output: The reconstructed input pattern that triggered these activations, allowing us to interpret what the feature represents.

Layer 0 - Feature 0

Layer 0 - Feature 0 Activation

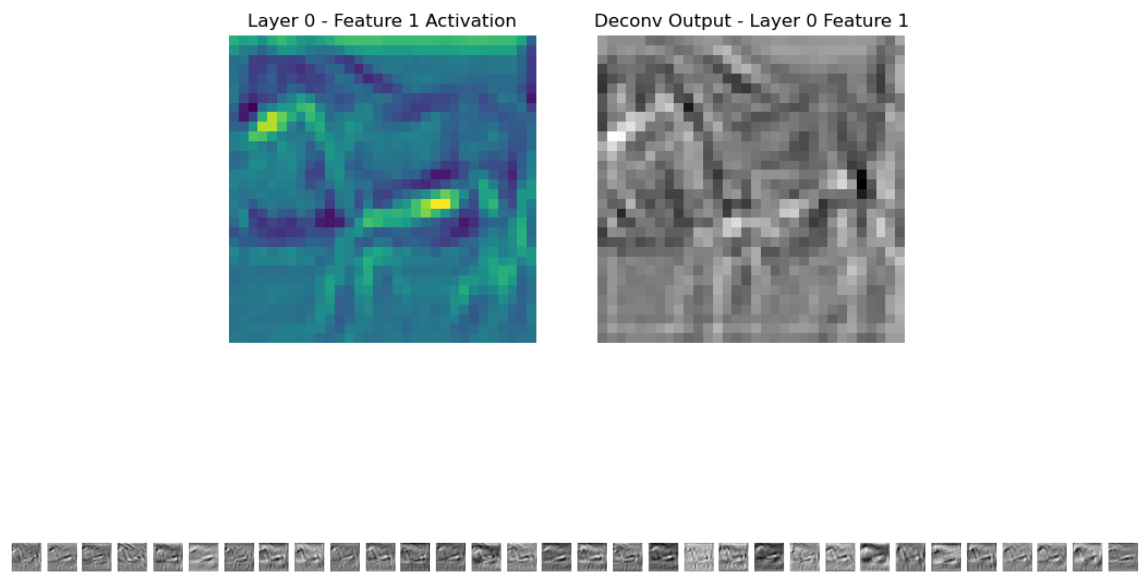


Deconv Output - Layer 0 Feature 0



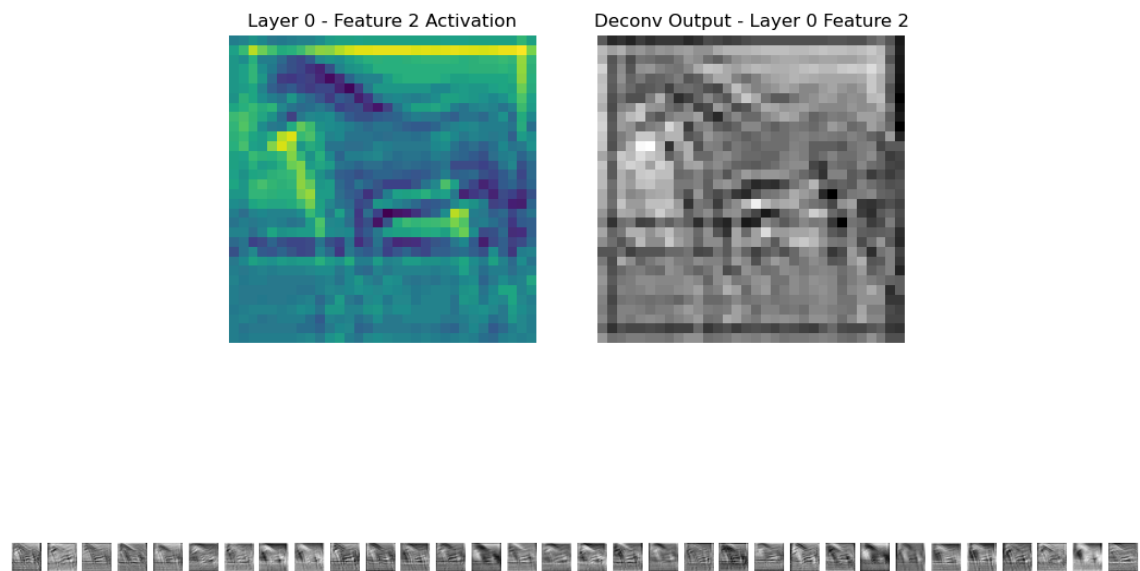
Feature 0 detects basic edges or gradients. The deconvolution output shows patterns in the input image that likely activate this feature map, which appear to correspond to areas with strong contrasts, typical of low-level edge detection.

Layer 0 - Feature 1



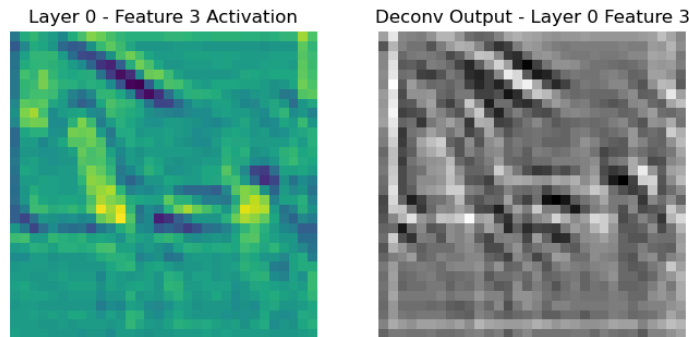
Feature 1 seems to capture another set of edge structures, possibly oriented differently than Feature 0. The deconvolution output highlights specific regions in the input where this feature map activates, focusing on line or texture patterns.

Layer 0 - Feature 2



Feature 2 reveals more complex textures or intersections in patterns, captures slightly more complex spatial relationships compared to Features 0 and 1.

Layer 0 - Feature 3



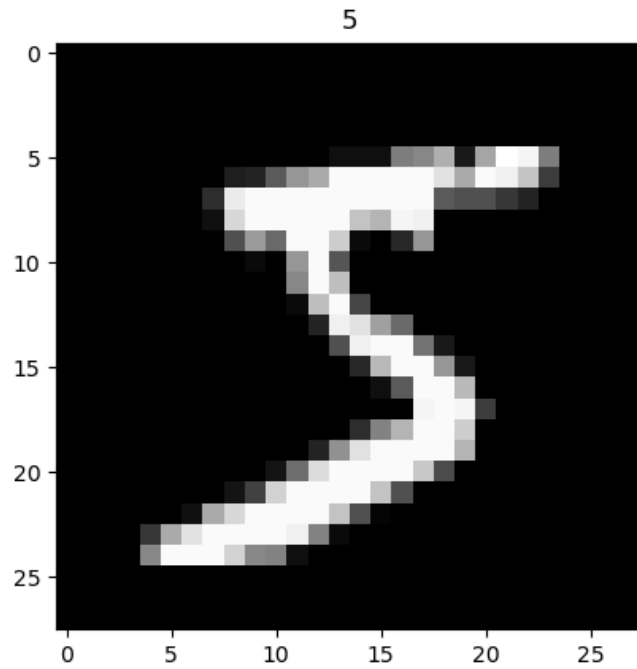
Feature 3 appears to detect further nuanced patterns, possibly related to textures or repetitive structures.

These visualizations show that the first convolutional layer focuses on capturing basic edge patterns and textures. Each feature map has a unique role, detecting various orientations, textures, and patterns that contribute to the model's understanding of image content.

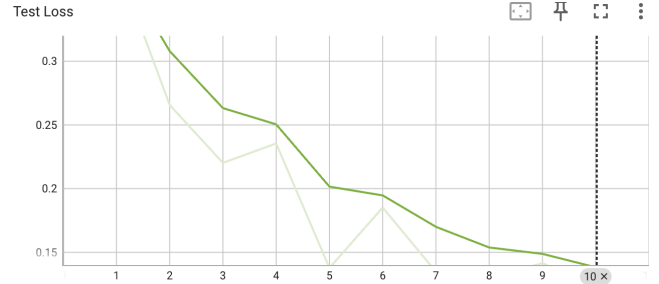
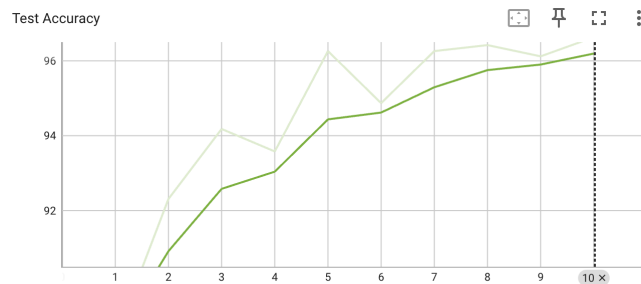
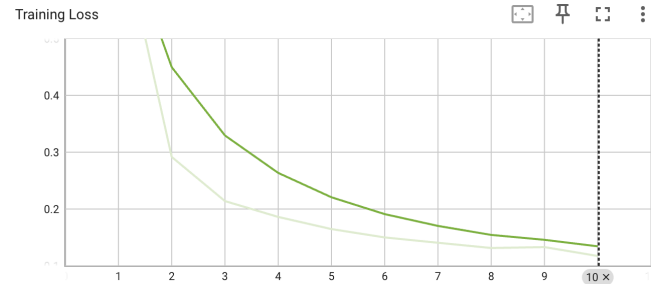
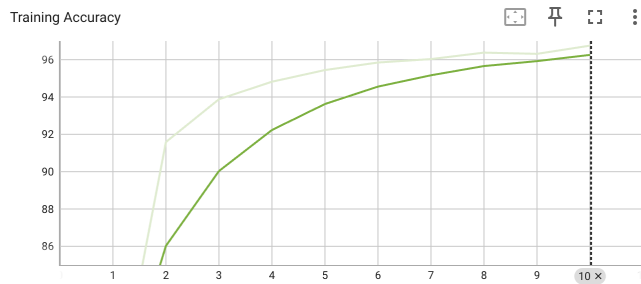
3. Build and Train an RNN on MNIST

3. a) Setup an RNN

plot one example:



To get better results, the learning rate is set to $lr = 0.001$. Below are the training/test accuracy and training loss plots generated:



Execution output:

```
% python assignment_2_part_2_rnn_mnist_skeleton.py
torch.Size([60000, 28, 28])
torch.Size([60000])
2024-11-05 13:43:09.134 python[68822:17633609] +[IMKClient subclass]: chose IMKClient_L
2024-11-05 13:43:09.134 python[68822:17633609] +[IMKInputSession subclass]: chose IMKIn
2024-11-05 13:43:12.574 python[68822:17633609] The class 'NSSavePanel' overrides the me
Starting Epoch 1/10
Training set: Average loss: 0.7130, Accuracy: 46052/60000 (76.75%)
Test set: Average loss: 0.3785, Accuracy: 8859/10000 (89%)
Epoch 1 Complete: Train Loss = 0.7130, Train Accuracy = 76.75%, Test Accuracy = 88.59%

Starting Epoch 2/10
Training set: Average loss: 0.2917, Accuracy: 54953/60000 (91.59%)
Test set: Average loss: 0.2657, Accuracy: 9231/10000 (92%)
Epoch 2 Complete: Train Loss = 0.2917, Train Accuracy = 91.59%, Test Accuracy = 92.31%

Starting Epoch 3/10
Training set: Average loss: 0.2140, Accuracy: 56327/60000 (93.88%)
Test set: Average loss: 0.2202, Accuracy: 9418/10000 (94%)
Epoch 3 Complete: Train Loss = 0.2140, Train Accuracy = 93.88%, Test Accuracy = 94.18%

Starting Epoch 4/10
Training set: Average loss: 0.1861, Accuracy: 56891/60000 (94.82%)
Test set: Average loss: 0.2354, Accuracy: 9358/10000 (94%)
Epoch 4 Complete: Train Loss = 0.1861, Train Accuracy = 94.82%, Test Accuracy = 93.58%

Starting Epoch 5/10
Training set: Average loss: 0.1647, Accuracy: 57266/60000 (95.44%)
Test set: Average loss: 0.1377, Accuracy: 9626/10000 (96%)
Epoch 5 Complete: Train Loss = 0.1647, Train Accuracy = 95.44%, Test Accuracy = 96.26%

Starting Epoch 6/10
Training set: Average loss: 0.1500, Accuracy: 57508/60000 (95.85%)
Test set: Average loss: 0.1852, Accuracy: 9487/10000 (95%)
Epoch 6 Complete: Train Loss = 0.1500, Train Accuracy = 95.85%, Test Accuracy = 94.87%

Starting Epoch 7/10
Training set: Average loss: 0.1407, Accuracy: 57619/60000 (96.03%)
Test set: Average loss: 0.1347, Accuracy: 9626/10000 (96%)
Epoch 7 Complete: Train Loss = 0.1407, Train Accuracy = 96.03%, Test Accuracy = 96.26%

Starting Epoch 8/10
Training set: Average loss: 0.1314, Accuracy: 57826/60000 (96.38%)
```

Test set: Average loss: 0.1301, Accuracy: 9642/10000 (96%)

Epoch 8 Complete: Train Loss = 0.1314, Train Accuracy = 96.38%, Test Accuracy = 96.42%

Starting Epoch 9/10

Training set: Average loss: 0.1330, Accuracy: 57787/60000 (96.31%)

Test set: Average loss: 0.1414, Accuracy: 9612/10000 (96%)

Epoch 9 Complete: Train Loss = 0.1330, Train Accuracy = 96.31%, Test Accuracy = 96.12%

Starting Epoch 10/10

Training set: Average loss: 0.1174, Accuracy: 58051/60000 (96.75%)

Test set: Average loss: 0.1226, Accuracy: 9663/10000 (97%)

Epoch 10 Complete: Train Loss = 0.1174, Train Accuracy = 96.75%, Test Accuracy = 96.63%

Over the 10 epochs, the model demonstrates significant improvements in both training and test performance. In the first epoch, the training accuracy reaches 76.75% with a test accuracy of 88.59%. As training progresses, both accuracy and loss continue to improve, with the model achieving a final training accuracy of 96.75% and a test accuracy of 96.63%.

This outcome suggests that with a learning rate of 0.001, the RNN is able to effectively learn the underlying patterns in the data, maintaining a stable and consistent convergence throughout training. The steady decrease in both training and test loss across epochs, along with the high final test accuracy, indicates that the model is well-optimized without significant issues related to vanishing gradients or overshooting.

3. b) How about using an LSTM or GRU

Continue using $lr = 0.001$:

LSTM Results:

Starting Epoch 1/10

Training set: Average loss: 0.4319, Accuracy: 51656/60000 (86.09%)

Test set: Average loss: 0.1571, Accuracy: 9550/10000 (96%)

Epoch 1 Complete: Train Loss = 0.4319, Train Accuracy = 86.09%, Test Accuracy = 95.50%

Starting Epoch 2/10

Training set: Average loss: 0.1209, Accuracy: 57838/60000 (96.40%)

Test set: Average loss: 0.1055, Accuracy: 9674/10000 (97%)

Epoch 2 Complete: Train Loss = 0.1209, Train Accuracy = 96.40%, Test Accuracy = 96.74%

Starting Epoch 3/10

Training set: Average loss: 0.0855, Accuracy: 58461/60000 (97.44%)

Test set: Average loss: 0.0741, Accuracy: 9782/10000 (98%)

Epoch 3 Complete: Train Loss = 0.0855, Train Accuracy = 97.44%, Test Accuracy = 97.82%

Starting Epoch 4/10

Training set: Average loss: 0.0636, Accuracy: 58816/60000 (98.03%)

Test set: Average loss: 0.0751, Accuracy: 9774/10000 (98%)

Epoch 4 Complete: Train Loss = 0.0636, Train Accuracy = 98.03%, Test Accuracy = 97.74%

Starting Epoch 5/10

Training set: Average loss: 0.0542, Accuracy: 58999/60000 (98.33%)

Test set: Average loss: 0.0626, Accuracy: 9814/10000 (98%)

Epoch 5 Complete: Train Loss = 0.0542, Train Accuracy = 98.33%, Test Accuracy = 98.14%

Starting Epoch 6/10

Training set: Average loss: 0.0447, Accuracy: 59149/60000 (98.58%)

Test set: Average loss: 0.0572, Accuracy: 9830/10000 (98%)

Epoch 6 Complete: Train Loss = 0.0447, Train Accuracy = 98.58%, Test Accuracy = 98.30%

Starting Epoch 7/10

Training set: Average loss: 0.0398, Accuracy: 59277/60000 (98.80%)

Test set: Average loss: 0.0527, Accuracy: 9844/10000 (98%)

Epoch 7 Complete: Train Loss = 0.0398, Train Accuracy = 98.80%, Test Accuracy = 98.44%

Starting Epoch 8/10

Training set: Average loss: 0.0352, Accuracy: 59361/60000 (98.94%)

Test set: Average loss: 0.0488, Accuracy: 9856/10000 (99%)

Epoch 8 Complete: Train Loss = 0.0352, Train Accuracy = 98.94%, Test Accuracy = 98.56%

Starting Epoch 9/10

Training set: Average loss: 0.0306, Accuracy: 59450/60000 (99.08%)

Test set: Average loss: 0.0467, Accuracy: 9859/10000 (99%)

Epoch 9 Complete: Train Loss = 0.0306, Train Accuracy = 99.08%, Test Accuracy = 98.59%

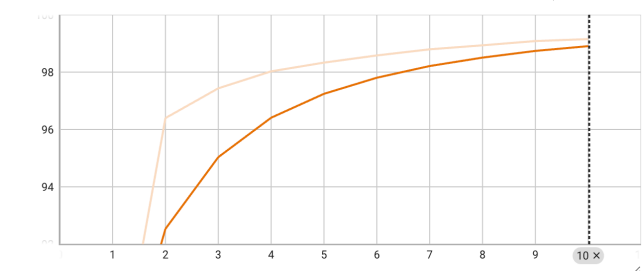
Starting Epoch 10/10

Training set: Average loss: 0.0284, Accuracy: 59491/60000 (99.15%)

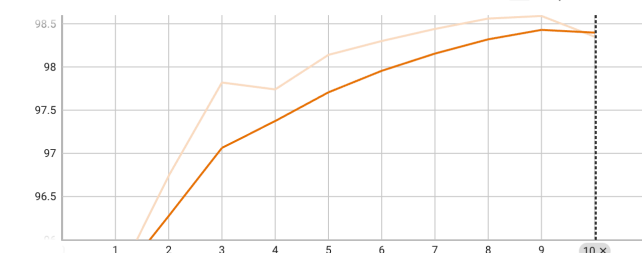
Test set: Average loss: 0.0604, Accuracy: 9835/10000 (98%)

Epoch 10 Complete: Train Loss = 0.0284, Train Accuracy = 99.15%, Test Accuracy = 98.35%

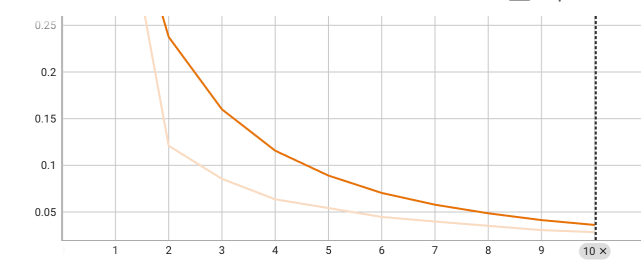
Training Accuracy



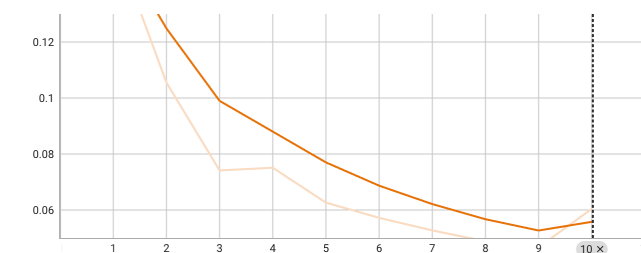
Test Accuracy



Training Loss



Test Loss



GRU Results:

Starting Epoch 1/10

Training set: Average loss: 0.4527, Accuracy: 51143/60000 (85.24%)

Test set: Average loss: 0.1395, Accuracy: 9584/10000 (96%)

Epoch 1 Complete: Train Loss = 0.4527, Train Accuracy = 85.24%, Test Accuracy = 95.84%

Starting Epoch 2/10

Training set: Average loss: 0.1057, Accuracy: 58097/60000 (96.83%)

Test set: Average loss: 0.0936, Accuracy: 9727/10000 (97%)

Epoch 2 Complete: Train Loss = 0.1057, Train Accuracy = 96.83%, Test Accuracy = 97.27%

Starting Epoch 3/10

Training set: Average loss: 0.0738, Accuracy: 58639/60000 (97.73%)

Test set: Average loss: 0.0649, Accuracy: 9797/10000 (98%)

Epoch 3 Complete: Train Loss = 0.0738, Train Accuracy = 97.73%, Test Accuracy = 97.97%

Starting Epoch 4/10

Training set: Average loss: 0.0561, Accuracy: 58971/60000 (98.28%)

Test set: Average loss: 0.0519, Accuracy: 9843/10000 (98%)

Epoch 4 Complete: Train Loss = 0.0561, Train Accuracy = 98.28%, Test Accuracy = 98.43%

Starting Epoch 5/10

Training set: Average loss: 0.0450, Accuracy: 59157/60000 (98.59%)

Test set: Average loss: 0.0547, Accuracy: 9831/10000 (98%)

Epoch 5 Complete: Train Loss = 0.0450, Train Accuracy = 98.59%, Test Accuracy = 98.31%

Starting Epoch 6/10

Training set: Average loss: 0.0372, Accuracy: 59321/60000 (98.87%)

Test set: Average loss: 0.0600, Accuracy: 9823/10000 (98%)

Epoch 6 Complete: Train Loss = 0.0372, Train Accuracy = 98.87%, Test Accuracy = 98.23%

Starting Epoch 7/10

Training set: Average loss: 0.0326, Accuracy: 59391/60000 (98.98%)

Test set: Average loss: 0.0471, Accuracy: 9863/10000 (99%)

Epoch 7 Complete: Train Loss = 0.0326, Train Accuracy = 98.98%, Test Accuracy = 98.63%

Starting Epoch 8/10

Training set: Average loss: 0.0271, Accuracy: 59480/60000 (99.13%)

Test set: Average loss: 0.0497, Accuracy: 9854/10000 (99%)

Epoch 8 Complete: Train Loss = 0.0271, Train Accuracy = 99.13%, Test Accuracy = 98.54%

Starting Epoch 9/10

Training set: Average loss: 0.0253, Accuracy: 59508/60000 (99.18%)

Test set: Average loss: 0.0393, Accuracy: 9882/10000 (99%)

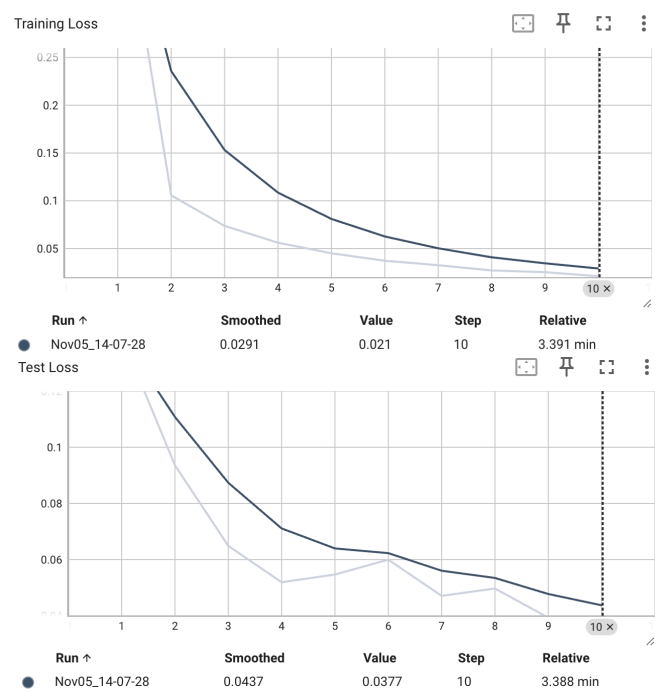
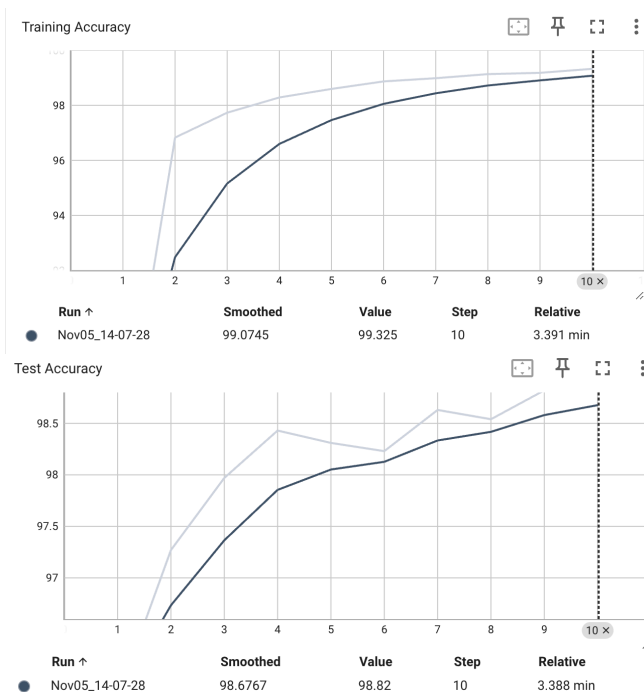
Epoch 9 Complete: Train Loss = 0.0253, Train Accuracy = 99.18%, Test Accuracy = 98.82%

Starting Epoch 10/10

Training set: Average loss: 0.0210, Accuracy: 59595/60000 (99.33%)

Test set: Average loss: 0.0377, Accuracy: 9882/10000 (99%)

Epoch 10 Complete: Train Loss = 0.0210, Train Accuracy = 99.33%, Test Accuracy = 98.82%



Accuracy Comparison:

- **RNN:** As mentioned in section 3.a), the RNN model reaches a training accuracy of 76.75% and test accuracy of 88.59% in the first epoch. Throughout the training process, accuracy continues to improve, with the model achieving final accuracies of 96.75% for training and 96.63% for testing by the end of 10 epochs. This demonstrates the model's ability to learn meaningful patterns with the revised learning rate (rather than $lr = 0.01$ with only about 10% accuracy).
- **LSTM:** The LSTM model starts with strong performance, achieving a training accuracy of 86.09% and a test accuracy of 95.50% in the first epoch. With each subsequent epoch, the model shows steady improvement, reaching a final training accuracy of 99.15% and test accuracy of 98.35%. This demonstrates that the LSTM is highly effective at capturing patterns in the data with high stability across epochs.
- **GRU:** The GRU model also exhibits strong initial performance, with a training accuracy of 85.24% and test accuracy of 95.84% in the first epoch. Like the LSTM, the GRU model continues to improve with each epoch, culminating in a final training accuracy of 99.33% and test accuracy of 98.82%, making it slightly more accurate on the test set compared to the LSTM.

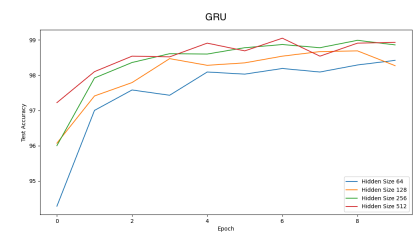
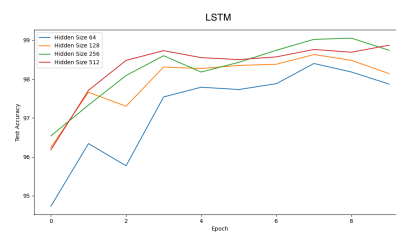
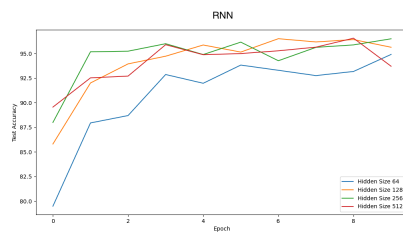
Loss Comparison:

- **RNN:** The RNN model exhibited a consistent decline in loss over the epochs, with the training loss decreasing from 0.7130 to 0.1174 and the test loss reducing from 0.3785 to 0.1226. This steady loss reduction reflects a well-converging model capable of effectively learning features over time.
- **LSTM and GRU:** Both LSTM and GRU models showed rapid and significant loss reductions in the initial epochs, with training and test losses staying low and relatively stable afterward. The final LSTM losses were 0.0284 for training and 0.0604 for testing, while the GRU ended with 0.0210 for training and 0.0377 for testing. These results illustrate that LSTM and GRU adapt well to the data, achieving even lower final losses than the RNN.

Change the number of hidden units

The results for different hidden sizes (64, 128, 256, 512) are shown in the table below:

Model Type	Hidden Size	Train Accuracy (Final)	Test Accuracy (Final)	Train Loss (Final)	Test Loss (Final)
RNN	64	94.50%	94.90%	0.1981	0.1823
	128	96.67%	95.63%	0.1177	0.1645
	256	96.28%	96.49%	0.1299	0.1317
	512	95.63%	93.70%	0.1513	0.224
LSTM	64	98.72%	97.87%	0.0427	0.0739
	128	99.21%	98.14%	0.026	0.0623
	256	99.40%	98.74%	0.0196	0.0415
	512	99.43%	98.87%	0.018	0.0366
GRU	64	98.83%	98.42%	0.0378	0.0557
	128	99.32%	98.27%	0.0208	0.0579
	256	99.46%	98.86%	0.0165	0.0411
	512	99.55%	98.93%	0.0146	0.0442



Analysis of Results:

- **RNN:** Increasing the number of hidden units noticeably improves the RNN's performance, with training and test accuracy reaching 94.50% and 94.90%, respectively, at 64 hidden units. However, as the hidden size increases, the improvements plateau, and test accuracy fluctuates. This highlights that while RNNs can learn some patterns at lower hidden sizes, they may struggle with further complexity on this task, especially compared to other models.
- **LSTM:** LSTM demonstrates high effectiveness across all hidden sizes, with final test accuracy consistently around or above 97%. At 256 and 512 hidden units, it reaches peak performance, achieving up to 98.87% test accuracy. The model converges well, and larger hidden sizes (128 and above) lead to faster and more stable improvements, showcasing LSTM's strong capacity for capturing complex sequential dependencies.
- **GRU:** GRU also performs well, with its best test accuracy reaching 99.05% at 512 hidden units. Performance is stable across hidden sizes, and like the LSTM, GRU benefits from increased hidden units, although its gains taper slightly beyond 256 units. The GRU's performance closely aligns with the LSTM, proving effective even with fewer parameters in smaller hidden configurations.

Summary:

- LSTM and GRU models substantially outperform the RNN, showing they are far more suited to handling sequential data and maintaining performance across varying hidden sizes.
- Higher hidden units generally boost performance, yet excessive parameters may lead to diminishing returns, making it necessary to balance model complexity and training stability.

3. c) Compare against the CNN

Compare with training using convnet in assignment 1:

1. Architecture:

- **CNN:** The CNN model from Assignment 1 is a deep convolutional network with four layers, specifically designed for image recognition tasks like MNIST. CNNs excel at capturing spatial hierarchies in data through convolutional filters, making them highly effective at extracting features from image data.

- **RNN:** Although RNNs are generally optimized for sequential data, here they process each MNIST image as a flattened sequence of pixels, allowing the model to capture some level of pixel dependencies. However, RNNs are not inherently suited for spatial data, meaning they might miss spatial hierarchies captured more effectively by CNNs.

2. **Accuracy:**

- **CNN:** In Assignment 1, the CNN achieved up to 99% accuracy on the MNIST dataset, highlighting its effectiveness in learning spatial features essential for image classification.
- **RNN:** With a learning rate of 0.001, the RNN model achieved a test accuracy of 96.63% by the end of 10 epochs. While still slightly below the CNN's accuracy, this result demonstrates a notable improvement and shows that RNNs can achieve competitive performance on MNIST, despite being less suited for image tasks.

3. **Convergence:**

- **CNN:** CNNs typically converge faster on image data, as convolutional layers efficiently capture important features with fewer parameters, allowing the model to learn effectively in fewer epochs.
- **RNN:** The RNN in this experiment reached competitive accuracy within 10 epochs, though it required careful tuning (e.g., setting a learning rate of 0.001) to stabilize and achieve rapid improvement. Compared to the CNN, the RNN converged slightly slower initially, but it managed to reach a high accuracy within a similar timeframe after optimization.

4. **Stability:**

- **CNN:** CNNs demonstrate stable training on image data, avoiding issues such as vanishing or exploding gradients, thanks to the local receptive fields of convolutional and pooling layers.
- **RNN:** RNNs may face stability challenges, especially on longer sequences, due to issues like vanishing or exploding gradients. However, in this experiment, with an optimized learning rate, the RNN achieved stable training and avoided major gradient issues.

In conclusion, although CNNs have an inherent advantage in spatial feature extraction for image classification tasks like MNIST, the RNN with optimized parameters achieved competitive performance, converging effectively and reaching a high accuracy. This highlights the flexibility of RNNs when properly tuned, though CNNs still maintain an edge in handling image data due to their ability to naturally capture spatial structures.